SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Tamil Elakiya S	**Reg. No.:**	192524443
Date:	17/08/2026	**Duration:**	60 minutes

Code of Conduct

I **Tamil Elakiya S (192524443)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Tamil Elakiya S**

Rubrics for Calculation

Criteria	Mark
System Requirement Analysis (30%)	**/5**
Selection of Software Architecture (25%)	**/5**
Architecture Diagram and Component Design (20%)	**/5**
Component Interaction and Quality Attribute Analysis (15%)	**/5**
Architectural Justification and Technical Presentation (10%)	**/5**
TOTAL	**/25**

Digital farmer Marketplace for Agricultural Product Trading

1. Problem Analysis

1.1 Problem Statement

The **Digital Farmer Marketplace for Agricultural Product Trading** is a web-based application designed to connect farmers directly with buyers. In the traditional agricultural trading system, farmers often depend on intermediaries to sell their products. This can reduce their profit and increase the final price paid by consumers.

The proposed system provides a centralized digital marketplace where farmers can register, list agricultural products, manage their inventory, and receive orders. Buyers can register, search for products, compare prices, place orders, and make payments through the platform.

The system aims to make agricultural trading more transparent, accessible, efficient, and convenient.

1.2 Project Objectives

The main objectives are:

- To provide a digital platform for direct farmer-to-buyer trading.
- To allow farmers to list and manage agricultural products.
- To enable buyers to search and purchase products online.
- To provide secure user authentication and transaction management.
- To maintain digital records of products, orders, and payments.
- To reduce dependency on intermediaries.
- To provide an easy-to-use and scalable application.

1.3 Functional Requirements

The major functional requirements include:

1. User registration and login.
2. Farmer profile management.
3. Buyer profile management.
4. Agricultural product listing.
5. Product search and filtering.
6. Inventory management.

7. Shopping cart management.
8. Order placement.
9. Payment processing.
10. Order tracking.
11. Notifications.
12. Reviews and ratings.
13. Administrator management.
14. Report generation.

2. Project Structure Design

2.1 Proposed Repository Structure

A suitable project structure for the Digital Farmer Marketplace is:

```
digital-farmer-marketplace/
│
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── assets/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── Dockerfile
│
├── backend/
│   ├── routes/
│   ├── models/
│   ├── controllers/
│   ├── services/
│   ├── middleware/
│   ├── config/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── database/
│   ├── schema.sql
│   └── seed.sql
│
├── docs/
```

```
│   ├── architecture/
│   ├── diagrams/
│   └── screenshots/
│
├── tests/
│   ├── frontend/
│   └── backend/
│
├── .gitignore
├── .env.example
├── README.md
├── docker-compose.yml
└── LICENSE
```

2.2 Purpose of Major Folders

frontend/

Contains all components related to the user interface.

It includes:

- Farmer dashboard
- Buyer dashboard
- Product pages
- Login and registration pages
- Shopping cart
- Order pages

backend/

Contains the server-side application and business logic.

It handles:

- Authentication
- Product management
- Orders
- Payments
- APIs
- Database communication

database/

Contains database-related files such as table definitions and sample data.

tests/

Contains automated and manual testing-related files.

docs/

Stores project documentation, architecture diagrams, workflow diagrams, and screenshots.

2.3 Important Files

File	Purpose
README.md	Provides project information and setup instructions
.gitignore	Prevents unnecessary/sensitive files from being committed
.env.example	Shows required environment variables without exposing secrets
docker-compose.yml	Defines and runs multiple containers
Dockerfile	Defines how an application container is built
requirements.txt	Lists Python backend dependencies
package.json	Defines frontend dependencies and scripts
schema.sql	Defines database structure

3. Git Repository Initialization

3.1 Introduction

Git is a distributed version control system used to track changes in source code. It allows developers to maintain different versions of the project, collaborate with team members, and recover previous versions when required.

For the Digital Farmer Marketplace, Git provides systematic management of frontend, backend, database, documentation, and Docker configuration files.

3.2 Repository Initialization Steps

After creating the project folder, Git can be initialized using:

git init

The project files are then added:

git add .

The first commit can be created using:

git commit -m "Initial project structure"

A remote GitHub repository can then be connected:

git remote add origin <repository-url>

The main branch can be pushed using:

git branch -M main

git push -u origin main

3.3 Essential Git Files

.git/

Created automatically by git init. It contains Git's internal repository information, including commits, branches, and configuration.

.gitignore

Specifies files that should not be tracked.

Example:

node_modules/

.env

__pycache__/

*.pyc

dist/

This is particularly important because passwords, API keys, and generated dependencies should not be uploaded to the repository.

README.md

Provides information about:

- Project purpose
- Technologies used
- Installation instructions
- Running instructions
- Docker commands

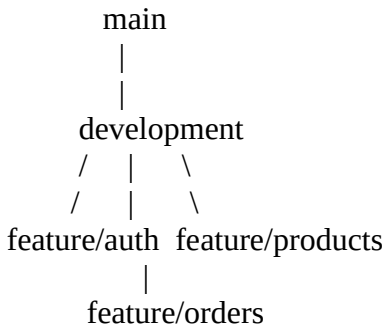
```
PS E:\Software engineering\Digital-Farmer-Marketplace> git add .
PS E:\Software engineering\Digital-Farmer-Marketplace> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   README.md
        new file:   backend/README.md
        new file:   database/schema.sql
        new file:   docs/README.md
        new file:   frontend/README.md
        new file:   tests/README.md

PS E:\Software engineering\Digital-Farmer-Marketplace> git commit -m "Initial project structur
e"
[main (root-commit) 4e0bead] Initial project structure
 7 files changed, 7 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 backend/README.md
 create mode 100644 database/schema.sql
 create mode 100644 docs/README.md
 create mode 100644 frontend/README.md
 create mode 100644 tests/README.md
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

Fig : Initialization of Git Repository

4. Git Branching Strategy

4.1 Proposed Branching Model

For this project, a simplified **Main–Development–Feature branching strategy** is suitable.



4.2 Main Branch

The main branch contains stable and tested versions of the application.

git checkout main

Only completed and verified features should be merged into this branch.

4.3 Development Branch

The development branch contains the latest integrated development version.

git checkout -b development

Features are merged into this branch before being released to main.

4.4 Feature Branches

Individual features are developed in separate branches.

Examples:

feature/user-authentication

feature/product-management

feature/order-management

feature/payment

feature/admin-dashboard

Example:

git checkout -b feature/product-management

4.5 Why This Strategy is Suitable

This strategy is suitable because:

- Different team members can work independently.
- Unfinished features do not affect the stable version.
- Features can be tested before integration.
- Development history remains organized.
- It reduces the possibility of introducing errors into main.

5. Version Control Workflow

The typical Git workflow for the project is:

Create Feature
↓
Create Feature Branch
↓
Write/Modify Code
↓
Test Application
↓
git add
↓
git commit
↓
Push Feature Branch
↓
Pull Request / Merge
↓
Development

↓

Testing

↓

Main

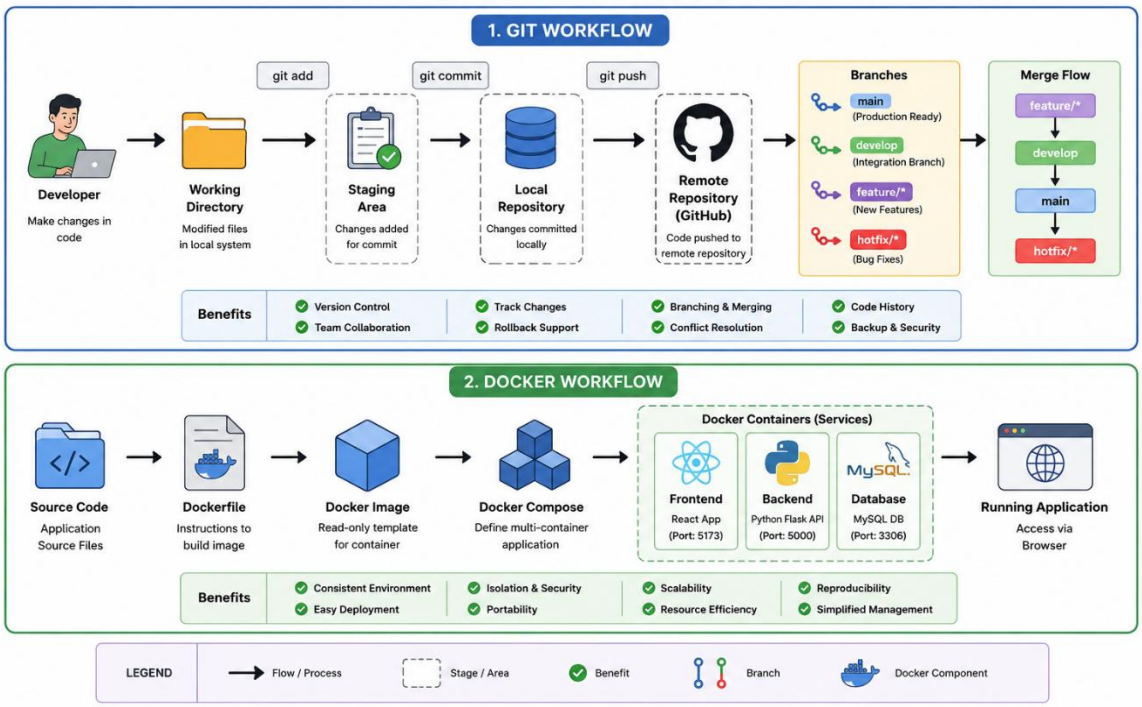


Fig : Workflow of Git and Docker

5.1 Creating a Feature Branch

- **git checkout development**
- **git pull origin development**
- **git checkout -b feature/product-management**

This creates an isolated workspace for product-related development.

```
PS E:\Software engineering\Digital-Farmer-Marketplace> git status
On branch feature/user-authentication
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory
)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

5.2 Staging Changes

- **git add .**

This places modified files into the staging area before committing.

5.3 Creating a Commit

- **git commit -m "Add product listing functionality"**

A commit creates a permanent record of the changes.

5.4 Synchronizing with GitHub

- **git push origin feature/product-management**

This uploads the branch and its commits to the remote repository.

5.5 Merging

After testing:

- **git checkout development**
- **git merge feature/product-management**

The completed feature is integrated into the development branch.

5.6 Conflict Resolution

If two branches modify the same portion of a file, Git may report a merge conflict.
The developer should:

1. Open the conflicting file.
2. Identify the conflicting sections.
3. Select or combine the correct changes.
4. Save the file.
5. Stage the resolved file.
   - **git add .**
   - **git commit -m "Resolve merge conflict"**

This ensures that the final version contains the correct implementation.

6. Commit History Analysis

6.1 Importance of Commit History

Commit history provides a chronological record of project development. It allows developers to understand what was changed, when it was changed, and why the change was introduced.

Example commit history:

Add Docker Compose configuration
Add order management module
Implement product search
Add farmer dashboard

Implement user authentication
Create database schema
Add initial project structure

6.2 Viewing Commit History

The following command displays the commit history:

git log --oneline

Example output:

a81f2c1 Add Docker Compose configuration
72bc910 Implement order management
43ad821 Add product search
29fc711 Add farmer dashboard
18de420 Implement authentication
09ab321 Create database schema

6.3 Good Commit Message Practices

Commit messages should be:

- Short
- Clear
- Meaningful
- Related to one logical change

Good Examples

Add farmer registration module
Fix product search validation
Update order status API
Add Docker configuration
Improve login error handling

Poor Examples

changes
update
final

new code
test

Meaningful commit messages make future maintenance easier.

7. Docker Environment Design

7.1 Introduction

Docker is a containerization platform that packages an application together with its dependencies into isolated containers. This ensures that the application behaves consistently across different development and deployment environments.

For the Digital Farmer Marketplace, Docker is useful because the application may contain multiple components such as the frontend, backend, and database.

7.2 Why Docker is Suitable

Docker provides:

- **Portability:** Application can run on different machines.
- **Consistency:** Development and deployment environments remain similar.
- **Isolation:** Each component can run independently.
- **Easy Deployment:** Containers can be started using a few commands.
- **Scalability:** Additional application containers can be created when required.
- **Dependency Management:** Required libraries and runtimes are packaged with the application.

7.3 Components Requiring Containerization

The proposed system can contain:

Frontend Container

↓

Backend/API Container

↓

Database Container

Additional containers can be added later for:

- Redis/cache
- Notification service
- Background worker

For the academic implementation, **Frontend + Backend + Database** are sufficient.

8. Dockerfile Development

8.1 Backend Dockerfile

For a Python/Flask backend, a sample Dockerfile is:

```
FROM python:3.12-slim

WORKDIR /app
```

```
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

8.2 Explanation

`FROM`
```
FROM python:3.12-slim
```

Provides the Python environment required to run the backend.

`WORKDIR`
```
WORKDIR /app
```

Sets `/app` as the working directory inside the container.

`COPY`
```
COPY requirements.txt .
```

Copies the dependency file into the container.

`RUN`
```
RUN pip install --no-cache-dir -r requirements.txt
```

Installs the required Python packages.

```
COPY . .
```

Copies the backend source code into the container.

`EXPOSE`
```
EXPOSE 5000
```

Documents the port used by the Flask application.

`CMD`
```
CMD ["python", "app.py"]
```

Starts the backend application.

8.3 Frontend Dockerfile

For a Vite/React frontend:

```
FROM node:20-alpine

WORKDIR /app
```

```
COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

This Dockerfile creates a Node.js environment, installs frontend dependencies, copies the source code, and starts the development server.

9. Docker Compose Design

9.1 Purpose

Docker Compose allows multiple containers to be defined and managed through a single configuration file.

A simplified docker-compose.yml can be:

```
services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: mysql://farmer_user:farmer_pass@database:3306/farmer_db
    depends_on:
      - database
```

```
PS E:\Software engineering\Digital-Farmer-Marketplace> docker compose up
farmer-marketplace-frontend  |   VITE v8.2.1  ready in 460 ms
farmer-marketplace-frontend  |
farmer-marketplace-frontend  |   ➜ Local:   http://localhost:5173/
farmer-marketplace-backend   | 172.19.0.1 - - [17/Aug/2026 15:33:58] "GET /api/products HTTP
/1.1" 200 -
farmer-marketplace-backend   | 172.19.0.1 - - [17/Aug/2026 15:33:58] "GET /favicon.ico HTTP/
1.1" 404 -

v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

```
  database:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: farmer_db
      MYSQL_USER: farmer_user
      MYSQL_PASSWORD: farmer_pass
      MYSQL_ROOT_PASSWORD: root_pass
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

If your actual application uses PostgreSQL instead of MySQL, the database service should be changed accordingly.

9.2 Services

Frontend

Runs the user interface.

Backend

Runs the API and business logic.

Database

Stores users, products, orders, and transaction information.

```
PS E:\Software engineering\Digital-Farmer-Marketplace> docker ps
place-frontend
d0e8cfb20d10   agriconnect-backend                 "python run.py"          3 days ago
       Up 16 minutes            0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   agriconnect-b
ackend
d7c8b96ef3a5   agriconnect-frontend                "docker-entrypoint.s…"   3 days ago
       Up 16 minutes            0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp   agriconnect-f
rontend
dbccae267846   mysql:8.4                           "docker-entrypoint.s…"   3 days ago
       Up 16 minutes (healthy)   3306/tcp                                      agriconnect-m
ysql
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

9.3 Networking

Docker Compose creates a network through which services can communicate using their service names.

For example:

backend → database:3306

The backend does not need to know the database container's IP address.

9.4 Volumes

volumes:
  mysql_data:

A volume ensures that database information remains available even if the database container is removed or recreated.

9.5 Environment Variables

Environment variables are used for configuration such as:

- Database name
- Database username
- Database password
- API keys
- Application settings

Sensitive values should preferably be stored in a .env file and excluded from Git using .gitignore.

10.Container Deployment and Testing

10.1 Building the Containers

From the project root:

docker compose build

This builds the required images.

10.2 Starting the Application

docker compose up

To run in the background:

docker compose up -d

10.3 Checking Running Containers

docker compose ps

Expected result:

NAME STATUS

frontend Running

backend Running

database Running

10.4 Testing the Application

The following tests can be performed:

Frontend Test

Open the frontend URL in a browser and verify that the marketplace interface loads correctly.

Backend Test

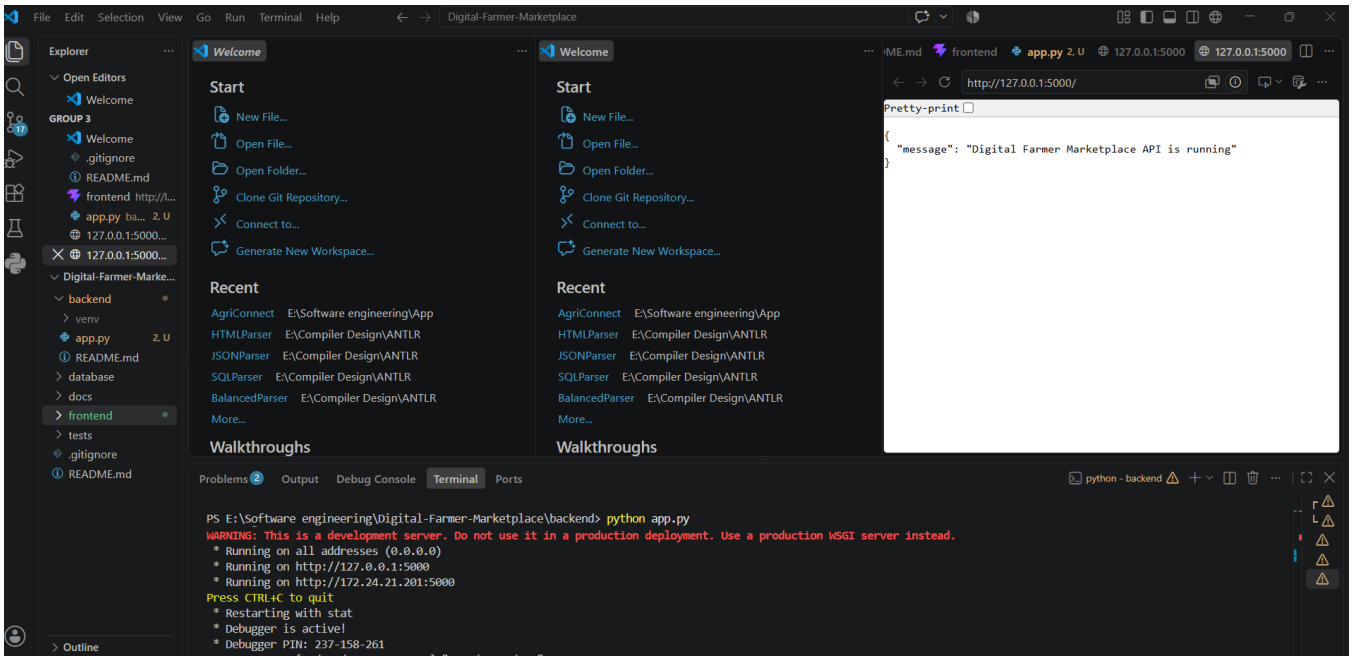
Verify that API endpoints respond correctly.

Example:

GET /api/products

POST /api/login

POST /api/orders



Database Test

Verify that:

- Users are stored.
- Products are stored.

- Orders are created.
- Database data persists after container restart.

10.5 Useful Docker Commands

- docker compose logs
- Displays application logs.
- docker compose restart
- Restarts the services.
- docker compose down
- Stops and removes containers.
- docker compose up --build
- Rebuilds images and starts the application.

11.Benefits of Git and Docker

11.1 Benefits of Git

Collaboration

Multiple developers can work on different features using separate branches.

Version Management

Every modification is recorded through commits, allowing previous versions to be restored.

Backup

The remote GitHub repository provides a centralized copy of the project.

Branch-Based Development

Features can be developed and tested independently.

Easy Maintenance

Commit history helps developers understand previous changes.

11.2 Benefits of Docker

Portability

The same containerized application can run across different environments.

Consistency

Docker reduces the "works on my machine" problem.

Deployment

The entire application can be started using:

docker compose up

Isolation

Frontend, backend, and database components operate in separate environments.

Scalability

Additional containers can be deployed when the system requires greater capacity.

Maintainability

Application dependencies and configurations are clearly defined through Dockerfiles and Compose files.

12.Challenges and Improvements

12.1 Challenges Encountered

Git Merge Conflicts

Conflicts may occur when multiple developers modify the same files.

Dependency Management

Different versions of Node.js, Python, or database software may cause compatibility issues.

Docker Configuration

Incorrect ports, environment variables, or service dependencies can prevent containers from starting.

Database Connectivity

The backend must use the correct Docker service name and database credentials.

Security

Passwords and API keys should not be directly committed to the Git repository.

12.2 Suggested Improvements

The project can be improved through:

- Automated GitHub Actions CI/CD.
- Automated testing before merging.
- Docker image optimization.
- Production-ready web server configuration.
- Secure secret management.
- Database backup automation.
- Container health checks.
- Monitoring and logging.
- Separate staging and production environments.
- Automated deployment to a cloud platform.

12.3 Future Enhancements

Future versions of the Digital Farmer Marketplace could include:

- Mobile application for farmers.
- Multilingual interface.
- AI-based product recommendations.
- Agricultural price prediction.
- Weather information.
- GPS-based delivery tracking.
- Digital payment expansion.
- Advanced sales analytics.
- Cloud-based deployment.
- Microservices architecture for large-scale expansion.

```
PS E:\Software engineering\Digital-Farmer-Marketplace> git log --oneline --graph --all -10
* c78e778 (HEAD -> main, origin/main, origin/development, development) Add Docker containerization for frontend and backend
* 7b09f0f (origin/feature/user-authentication, feature/user-authentication, feature/product-management) Add project overview and main features
* 4e0bead Initial project structure
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

Fig : Commit History Analysis

```
PS E:\Software engineering\Digital-Farmer-Marketplace> git branch -a
  development
  feature/product-management
  feature/user-authentication
* main
  remotes/origin/development
  remotes/origin/feature/user-authentication
  remotes/origin/main
PS E:\Software engineering\Digital-Farmer-Marketplace>
```

Fig : Git Branching Strategy